

```
import ipywidgets as widgets
import matplotlib.pyplot as plt
import numpy as np
```

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score
```

```
def update_kmeans(n_samples, n_clusters, cluster_std):
```

```
    # 1. Generate dynamic data
```

```
    X, y_true = make_blobs(
        n_samples=n_samples,
        centers=n_clusters,
        cluster_std=cluster_std,
        random_state=42
    )
```

```
    # 2. Train K-Means
```

```
    kmeans = KMeans(
        n_clusters=n_clusters,
        random_state=42,
        n_init=10
    )
```

```
    labels = kmeans.fit_predict(X)
```

```
    centers = kmeans.cluster_centers_
```

3. Calculate Silhouette Score

try:

```
score = silhouette_score(X, labels)
```

except:

```
score = 0.0
```

4. Create the graph

```
fig, ax = plt.subplots(figsize=(8, 6))
```

Plot each cluster individually

for i in range(n_clusters):

```
cluster_points = X[labels == i]
```

Check whether cluster number is even or odd

if i % 2 == 0:

```
color = '#1f77b4'
```

```
label_name = f'Cluster {i} (Even)'
```

```
marker_style = 'o'
```

else:

```
color = '#ff7f0e'
```

```
label_name = f'Cluster {i} (Odd)'
```

```
marker_style = 's'
```

```
ax.scatter(
```

```
    cluster_points[:, 0],
```

```
    cluster_points[:, 1],
```

```
    color=color,  
    marker=marker_style,  
    s=50,  
    edgecolor='k',  
    alpha=0.8,  
    label=label_name  
)
```

5. Plot centroids

```
ax.scatter(  
    centers[:, 0],  
    centers[:, 1],  
    marker='X',  
    s=250,  
    color='red',  
    edgecolor='white',  
    linewidth=2,  
    label='Centroids'  
)
```

6. Add title

```
ax.set_title(  
    f'K-Means Clustering\n'  
    f'Clusters (k): {n_clusters} | '  
    f'Silhouette Score: {score:.3f}',  
    fontsize=12  
)
```

```
ax.set_xlabel('Feature 1')
```

```
ax.set_ylabel('Feature 2')
```

```
ax.legend(  
    loc='upper right',  
    bbox_to_anchor=(1.25, 1)  
)
```

```
ax.grid(  
    True,  
    linestyle='--',  
    alpha=0.5  
)
```

```
plt.tight_layout()  
plt.show()
```

7. Interactive Widgets

```
widgets.interact(  
    update_kmeans,  
  
    n_samples=widgets.IntSlider(  
        min=100,  
        max=1000,  
        step=50,  
        value=300,  
        description='Samples:'  
    ),
```

```
n_clusters=widgets.IntSlider(
    min=2,
    max=8,
    step=1,
    value=4,
    description='Clusters (k):'
),

cluster_std=widgets.FloatSlider(
    min=0.4,
    max=3.0,
    step=0.1,
    value=1.0,
    description='Spread (Std):'
)
)
```